PaDeSA: Parametric Design Synthesis Algorithms

Lingqi Li, Sean Lin, Stanley Wang

(*All authors contributed equally to this manuscript)

***Abstract*—We present PaDeSA (Parametric Design Synthesis Algorithms) as an exploration of machine learning techniques in the domain of design and modeling. We begin by showing the critical role of feature selection and dimensionality reduction with 3D model data, specifically highlighting significance in constructing simple classifiers. Next, we present the development of a novel reinforcement learning approach to extract parametrized feature semantics from 3D data. Finally, we present ongoing work on applying deep learning with generative adversarial networks (GANs) to synthesize new 3D morphology from given distributions. These algorithms collectively underpin a framework for facilitating 3D modeling, encompassing initial shape conceptualization to developing complex parameterized models.**

***Index Terms*—3D Geometry Generation, Reinforcement Learning, Classification**

I. Introduction

3D object modeling is used widely in the field of engineering to synthesize geometric components for a wide range of tasks and applications. Typically, this is done in a CAD (computer-aided design) application such as Solidworks, Fusion360, Onshape, etc. Unlike other domains of 3D design (such as computer-generated imagery for video games) where triangular meshes are commonly used to capture complex forms, engineering components are designed parametrically through the sequential addition and subtraction of 2D sketches and 3D object primitives (box, sphere, cylinder, etc.). This lends an advantage in the sense that dimensions and tolerances can be defined exactly for the geometry, and the geometry is easily adjusted through modification of its key parameters (radius, length, depth, etc). An example of how a bolt could typically be constructed in a standard parametric design workflow is given in Fig. 1.

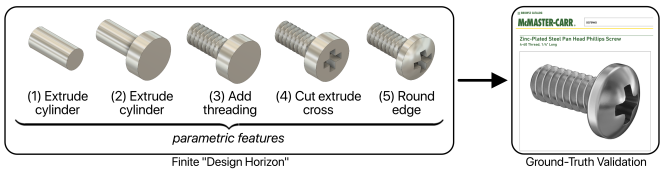

Fig. 1. Representative CAD workflow showing a sequence of parametrized operations to create a bolt

Our approach is divided into three parts: (1) representation of 3D model features (2) sequential parametric design (3) generation of novel geometries. For (1), we experiment with different features (area, volume, # of edges, etc.) as input to simple classifiers (decision tree, softmax log reg, naive bayes) for distinguishing between labels of nuts, bolts, and washers. For (2), we train a reinforcement learning algorithm that takes a given geometric part as input and finds a "design policy" or sequence of parametric features to best approximate the part design. For (3), we explore the setup of a generative adversarial network (GAN) to generate novel geometries from a distribution of voxel data.

II. Related Work

Much of the current work that lies at the intersection of machine learning and 3D model/scene representation has concerned the use of Neural Radiance Fields (NeRFs) [1]. These are fully connected non-convolutional deep networks that simulate camera rays in a continuous volumetric scene. Recent works such as DreamFusion [2] and Magic3D [3] have studied the generation of novel NeRFs using a text-to-image diffusion process. However, novel generation and classification of parametric designs common to engineering remains an under-explored topic. The ABC dataset [4] of geometric CAD parts has prompted several initial works on parametric feature recognition using deep learning [5, 6]. Nonetheless, there is still a large gap in the application of other machine learning techniques towards a robust and comprehensive framework for parametric 3D models.

III. Feature-Based Classification of Simple Engineering Components

A. Dataset

Our training and validation data are parametric STEP (Standard for the Exchange of Product Data) files extracted from McMaster Carr, a large supplier of industrial components. We consider classifying between three different labeled components: bolts, nuts, and washers with a total of 600 labeled samples (200 for each class). STEP files contain proprietary encoding of 3D model data and vary largely in size and complexity depending on the component. Thus, directly working with STEP data is undesirable. Conversion from STEP to a mesh, pointcloud, or voxel grid is also unideal due to the high dimensionality of these representations. Thus, we implement CADQuery (Python API) to extract key geometric features from our STEP models and effectively reduce the dimensionality necessary to interpret 3D objects. We extract a total of 15 features including X, Y, and Z dimensions of

the part's bounding box, the number of vertices/edges/faces, volume and surface area, etc. Additionally, 8 nonlinear features are considered as a combination of these original 15 features (such as nondimensional length and volume aspect ratios). The original 15 features are used in our comparison of different classifiers, while the 8 non-linear features are subsequently added to enhance our best classification model. An illustration of our data import and processing pipeline for classification is provided in Fig. 2, and a complete list of geometric features used for classification is provided in Table I.

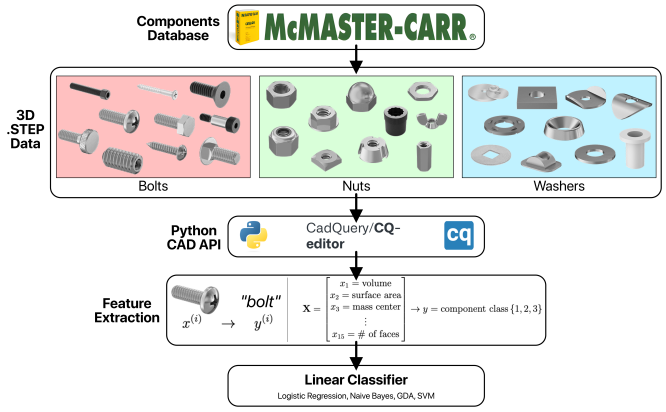

Fig. 2. Processing and feature extraction pipeline for training linear classifiers to distinguish between labeled components (bolts, nuts, & washers) from McMaster-Carr database

Feature #	Description
f_1	volume
f_2	surface area
f_3, f_4, f_5	bounding box dimensions (x, y, z)
f_6	bounding box volume
f_7, f_8, f_9	bounding box centroid (x, y, z)
f_{10}, f_{11}, f_{12}	center of mass (x, y, z)
f_{13}	number of vertices
f_{14}	number of edges
f_{15}	number of faces

TABLE I
GEOMETRIC FEATURES USED FOR CLASSIFICATION

B. Methods

We selected Softmax Regression, Decision Tree, and Gaussian Naive Bayes classifiers for our multi-class classification tasks. Softmax regression predicts the probability that a given data point belongs to a specific class. For a training sample x, the model first computes a score $s_k(x)$ for each class k. $s_k(x) = x^T\theta^{(k)}$ The softmax function for class k is defined as:

$$P(y = k|x) = \frac{\exp(s_k(x))}{\sum_{j=1}^{K} \exp(s_j(x))}$$

Decision trees are a non-parametric supervised learning method. It breaks down a dataset into smaller and smaller subsets while developing an associated decision tree incrementally. The final result is a tree with decision nodes and leaf nodes.

The Gaussian Naive Bayes classifier assumes that the continuous values associated with each feature are distributed according to a Gaussian distribution. The algorithm works on maximizing the posterior probability. It calculates the mean μ and variance σ^2 of each feature for each class. The probability of a feature given a class is modeled using the Gaussian function:

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right)$$

To train our models, we took 90% of the dataset as the training set and the other 10% as validation. In training our Softmax Regression model, we have 50000 epochs and the learning rate is set to 1e-5.

C. Results

We begin by comparing our three classification models on the original 15 geometric features. The accuracy of the Softmax Logistic Regression was found to be 36.67% as shown in Fig. 3. One potential cause for this unreliable accuracy is the linearity of the softmax decision boundaries, which may not fit our more complex linearly inseparable features. The Decision Tree model resulted in a reasonable accuracy of 82.22%, which is slightly lower than the performance of the Gaussian Naive Bayes Model. The Gaussian Naive Bayes achieved the highest accuracy of 83.33%, which indicates its capability to distinguish between the classes based on the Gaussian distribution of the features. Thus, we conduct further training on this model utilizing an updated dataset with more nonlinear features. As stated previously, these features are simply a nonlinear combination of the existing features, such as dividing two lengths to create e non-dimensional aspect ratio. The 8 new nonlinear features added to the dataset greatly improved the model's accuracy, which resulted in a test set accuracy as high as 88.33%.

Classification Model	Test Set Accuracy
Softmax Logistic Regression	36.67%
Decision Tree	82.22%
Gaussian Naive Bayes	83.33%
Gaussian Naive Bayes w/ nonlinear features	88.33%

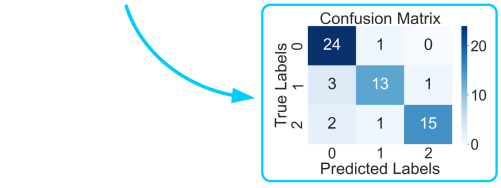

Fig. 3. Accuracies of various classifiers on the test dataset. A confusion matrix is provided for the highest performing model (0 = bolt, 1 = nut, 2 = washer).

We subsequently implement an ablation study with the Gaussian Naive Bayes model extended by nonlinear feature combinations (i.e. aspect ratios). This helps us to study the importance of each of our features. We find that certain features such as (A) raw length or (B) center of mass coordinate are insignificant and produce little change when ablated while other features such as $\frac{Y}{Z}$ aspect ratio (C) and face to vertex ratio (D) produce noticeable accuracy losses when ablated. The distributions of each feature across different labels is shown in Fig. 4, and the accuracies of the ablation study are given in Table II.

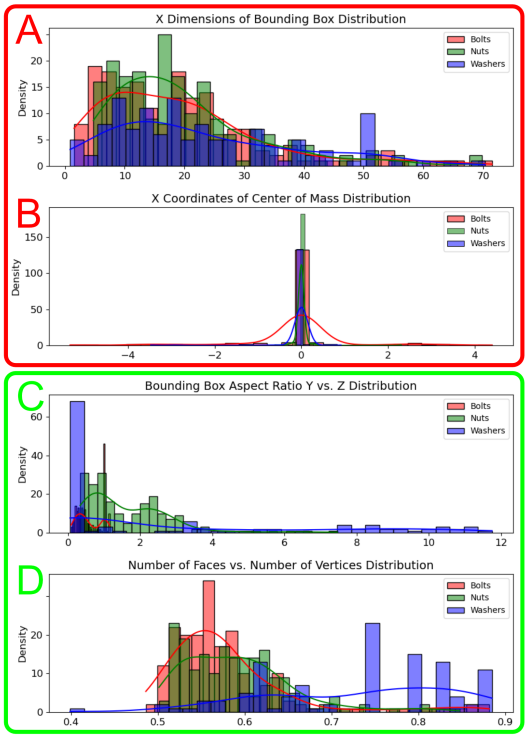

Fig. 4. Distributions for features used in the ablation study: (A) length (B) COM (C) $\frac{Y}{Z}$ aspect ratio (D) face to vertex ratio

Ablated Feature	Accuracy
(none)	88.33%
A	88.33%
B	88.33%
C	80.00%
D	86.67%

TABLE II
ACCURACY OF THE GAUSSIAN NAIVE BAYES CLASSIFIER AFTER ABLATION (REMOVAL) OF SELECT FEATURES (A,B,C,D)

IV. REINFORCEMENT LEARNING FOR SEQUENTIAL PARAMETRIC DESIGN

A. Dataset

The input to our reinforcement learning model is a voxel grid, which can be considered as a 3D binary occupancy grid. The output is a parametric design "policy" encoded as a STEP file. The reason for this choice of input and output is motivated by the fact that voxels are the most suitable flavor of 3D data for generative models and a common baseline for describing point clouds or meshes.

Validation objects for this RL model were manually sourced. The easy object consists of a pair of primitive bodies. The medium object consists of a screw sourced from McMaster Carr. The hard object consists of a complex star-like object. We attempt to learn the series of parametric design actions best suited for constructing these objects.

B. Methods

This problem is a sequential problem that can be structured as a Markov Decision Process (MDP). The state space $|S|$ of this MDP problem is a continuous 3D bounding box around the target object that we discretized into a binary voxel by partitioning to a fixed resolution. The input voxel representing the target object is set as the target state s_t.

The action space $|A|$ of this MDP problem is inspired by capabilities of common 3D object design software. Our action space consists of three primitive operations: create a box, cylinder aligned with the cardinal axes, or sphere. The centroids and dimensions of these objects are also inherently continuous parameters but discretized for the sake of this problem. A policy π is considered a series of actions to roll out from an empty starting state.

Finally, we define a reward function $R(s, a, s')$, where β is a hyperparameter in our definition and $\beta \leq -1$.

- if $s[x, y, z] == s_t[x, y, z] == 1$ then $R(s) = 1$. True positives are rewarded .
- if $s[x, y, z] == s_t[x, y, z] == 0$ then $R(s) = 0$. True negatives are not rewarded
- if $s[x, y, z] \neq s_t[x, y, z]$ then $R(s) = \beta$. False positives or negatives are penalized

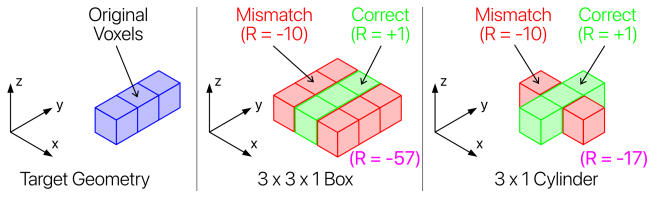

Fig. 5. Demonstration of how rewards are calculated based on a Target Geometry. $\beta = -10$

As defined, the current action space is the set of all possible boxes, cylinders, and spheres that can be created from any coordinate in the voxel. To shrink the size of our action space, we only decide on the next coordinates to grow from. The best object to grow from said coordinate is efficiently calculated via an inspired rollout algorithm [7].

An action exploration algorithm must be implemented to determine which coordinate to use as the next centroid for object growth. Our exploration algorithm takes the remaining coordinates of the target that need to be occupied and runs k-means with a variable cluster count k. The mean of the largest cluster is considered as the next centroid to explore. If the mean of the largest cluster is itself vacant (0 in the voxel),

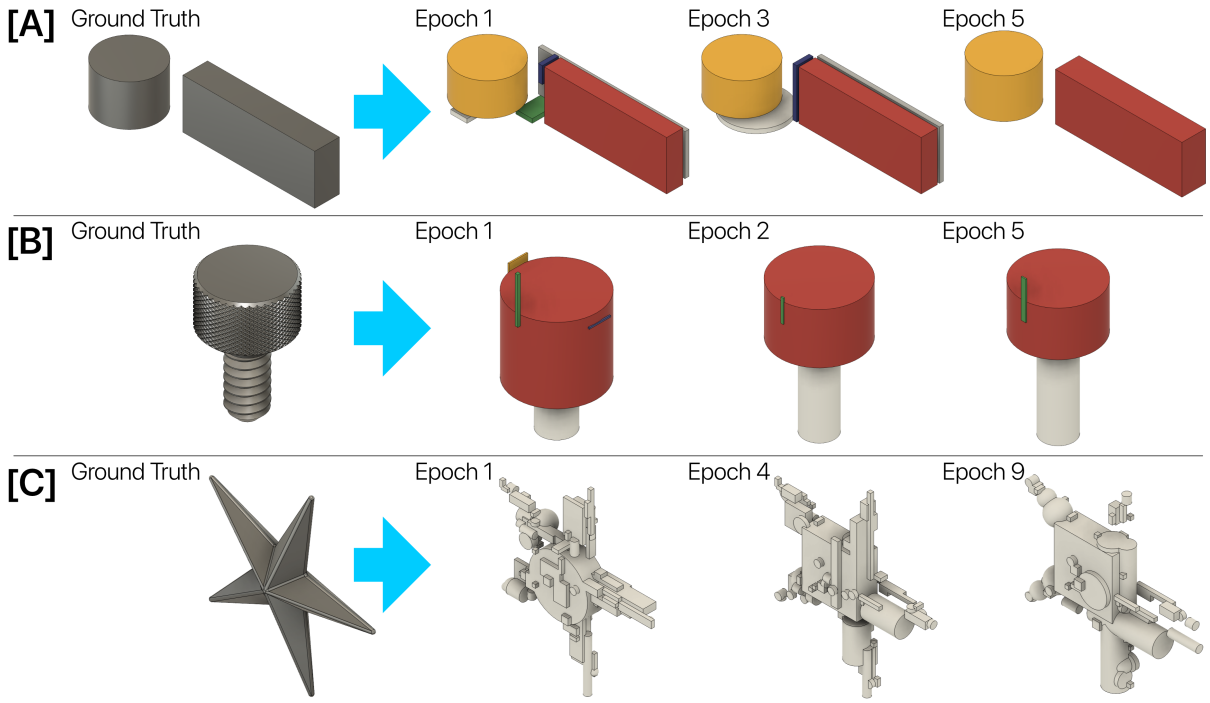

Fig. 6. Easy, medium, and hard objects at different stages of training epochs.

then a random occupied coordinate is chosen.

A policy needs a heuristic to know when it can stop adding actions (policy termination) as well as an iteration algorithm to tell it how to improve given a previous policy (policy iteration). The heuristic for policy termination is defined as when at least α proportion of the target voxel has been occupied.

For policy iteration, we use the Hooke-Jeeves algorithm paired with action pruning[8]. The Hooke-Jeeves algorithm is a flavor of gradient ascent for discrete action spaces with no defined underlying value function. For a given action in a policy, we take the centroid of the action and examine its 6 neighbors with a starting step size. If the best object that can be rolled out from a neighbor coordinate is better than the current object being rolled out from this coordinate, that action in the policy is updated to be the new action from the neighboring coordinate. If no neighbors are better at any shrinking step size, then the status-quo action is retained.

If an updated action in the policy now occupies the centroid from which a future action grows from, the future action is removed from the policy (pruned). Policies will iterate until a convergence criteria is reached. The below equation is our definition of policy valuation.

$$V^{\pi}(s_i|a_i) = R(s_i) + \gamma V^{\pi}(s'|a')$$

This equation if a flavor of the Bellman Equation tailored for our MDP problem and places a value on a policy π. s_i, a_i are the initial state and first action in the policy and γ is the discount rate of future state-action rewards. Policy iteration is defined as converged when $\frac{\Delta V^{\pi}}{V^{\pi}} < \epsilon$ where ϵ is our final hyperparameter.

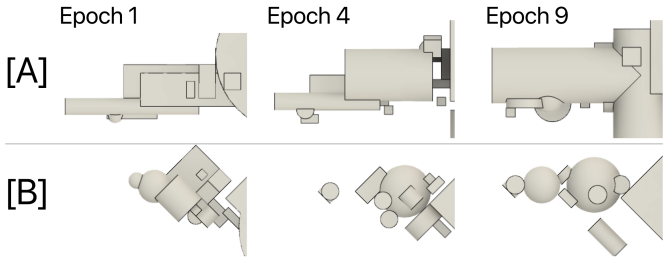

Fig. 7. Policy evolution over 9 epochs for parametrizing two different "legs" of the complex star-shaped object

C. Results

We begin by discussing the results of the RL model when learning how to make objects of varying difficulty with a fixed set of hyperparameters. Then we discuss how tuning each of the hyperparameters impacts the results.

Fig. 6 illustrates the policy evolution of the RL model for an easy, medium, and hard object. Parameters for these tests were as follows: ($\beta = 10$, $\alpha = 0.95$, $\gamma = 0.95$, and $\epsilon = 0.01$). Immediately, we notice that Hooke-Jeeves policy iteration certainly converges to an end result that matches the target object closer and closer. Upon close inspection, it may also be observed that the number of actions taken (indicated by the number of primitive bodies in the object) reduces over multiple epochs.

1) Action count trajectory of Easy Object: $6 \rightarrow 5 \rightarrow 3 \rightarrow 2 \rightarrow 2$
2) Action count trajectory of Medium Object: $5 \rightarrow 4 \rightarrow 3 \rightarrow 3 \rightarrow 3$

3) Action count trajectory of Hard Object: $91 \rightarrow 80 \rightarrow 76 \rightarrow 71 \rightarrow ... \rightarrow 54$

Finally, an interesting observation can be made on the hard object especially. Referring to Fig. 7, the RL model seems to learn the most efficient way to create arms of the star using available actions while minimizing the number of actions. For the bottom left arm [B] which is unaligned with any cardinal axes, the RL model learns to create efficiently using a series of shrinking spheres. For the bottom right arm [A] which is loosely aligned with a cardinal axis, the RL model learns to create efficiently using a cylinder aligned with said cardinal axis.

Lastly, we discuss the impact of each hyperparameter on the performance of the RL model.

α simply measures how much of the target object's space occupancy needs to be filled for our output to be considered complete. Ideally, $\alpha = 1$ as that means the output object must fully encompass the target object. In practice, this leads to drastic extensions in model convergence time.

ϵ is also a simple parameter that determines how similar two successive learned policies must be in order for the policy to be considered converged. Ideally, $\epsilon = 0$ as that means the successive policies were identical and the policy has converged. However, this also leads to drastic convergence time extensions and potential infinite loops.

β and γ are stickier parameters that have a large impact on the output produced as well as on each other.

1) For $\gamma \rightarrow 0$ and $\beta << -1$: The model prioritizes obtaining as much reward as possible on earlier actions, yet it also wants to fit the target object as tightly as possible. This leads to conflicting interests in the model.
2) For $\gamma \rightarrow 0$ and $\beta \rightarrow -1$: The model prioritizes obtaining as much reward as possible on earlier actions, and it also allows for looseness in how nicely an action needs to fit the target object. This leads to agreeing interests in the model and a low number of loose-fitting actions.
3) For $\gamma \rightarrow 1$ and $\beta \rightarrow -1$: The model does not discount an extended number of actions and is only focused on fitting the model well, yet also allows for looseness in how nicely an action needs to fit the target object. This leads to conflicting interests in the model.
4) For $\gamma \rightarrow 1$ and $\beta << -1$: The model does not discount an extended number of actions and is only focused on fitting the model well, and it also wants to fit the target object as tightly as possible. This leads to agreeing interests in the model and a very high number of tight-fitting actions.

V. Generative Adverserial Networks for Novel Shape Construction

A. Data

We used ABC dataset [4] for this model. We converted the stl. files to 50 x 50 x 50 resolution voxel grid.

B. Methods

We proposed utilizing Generative Adversarial Networks (GANs) to generate novel three-dimensional representations. We hope to develop a model capable of learning the distribution of engineering component shapes in the ABC dataset within a 3D shape. The training data is normalized to a range of [-1,1]. The input to our GAN is a noise vector and the output is a voxel grid. We designed our GAN with a generator and discriminator architecture. The generator network starts with a dense foundation and expands through 3D transposed convolutions to output the final voxel grid [9]. The discriminator uses 3D convolutions to distinguish between real and generated data. The GAN was trained using an Adam optimizer with an adjustable learning rate.

C. Results

The outcome of our GANs model indicated a learning curve with the model initially producing a fully-filled grid, then gradually leading to a less occupied voxel grid, which shows progress with the continued training iterations. However, the model is currently still immature and a large amount of tuning is needed for further development.

VI. Conclusions and Future Work

We present PaDeSA: Parametric Design Synthesis Algorithms as a case study towards the application of machine learning in 3d engineering design. First, we show that proper selection of geometric features is critical for effective dimensionality reduction of complex 3D shape data. Examining data from McMaster Carr, we found a Gaussian Naive Bayes classifier to work best as a result of several important normally distributed features (such as the number of edges).

We continue by introducing the development of a reinforcement learning model for synthesizing design "policies", or the sequence of parametric actions necessary to construct an engineering component. We discretize the state and action spaces and define a hyper-parameterized reward function to capture the problem as a Markov Decision Process (MDP). Then we implement a exploration, rollout, and Hooke-Jeeves policy iteration algorithm to converge to the optimal set of actions as the design policy.

Lastly, we explore deep learning for novel shape generation. Our GAN model is still undergoing major development. However, our findings suggest that even if challenges persist in training and tuning the model, it holds potential for 3D data generation. For future work, we'll consider further tuning, implementing more complex loss functions, and integrating regularization methods to enhance the model's performance and stability.

VII. Contributions

Equal contributions were made in unique areas by all authors. Stanley Wang and Lingqi Li built, trained, and analyzed the feature-based classification model. They also attempted to build a GAN to generate novel voxel grids from training distributions. Sean Lin built, trained, and analyzed the reinforcement learning model. He also wrote auxiliary software for data processing and visualization. Equal contributions were made in writing the report.

VIII. Overlap with other course

Sean Lin will use parts of this project for AA 228. This is the LINK to the Google Drive where the final report of the other class will live.

References

[1] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, & Ren Ng. (2021). NeRF: representing scenes as neural radiance fields for view synthesis. *Communic. ACM*, 65(1), 99–106. https://doi.org/10.1145/3503250

[2] Poole, B., Jain, A., Barron, J. T., & Mildenhall, B. (2022). DreamFusion: Text-to-3D using 2D Diffusion. *ArXiv*. https://arxiv.org/abs/2209.14988

[3] Lin, C., Gao, J., Tang, L., Takikawa, T., Zeng, X., Huang, X., Kreis, K., Fidler, S., Liu, M., & Lin, T. (2022). Magic3D: High-Resolution Text-to-3D Content Creation. *ArXiv*. https://arxiv.org/abs/2211.10440

[4] Koch, S., Matveev, A., Jiang, Z., Williams, F., Artemov, A., Burnaev, E., Alexa, M., Zorin, D., & Panozzo, D. (2018). ABC: A Big CAD Model Dataset For Geometric Deep Learning. *ArXiv*. https://arxiv.org/abs/1812.06216

[5] Cao, W., Robinson, T., Hua, Y., Boussuge, F., Colligan, A. R., & Pan, W. (2020). Graph Representation of 3D CAD Models for Machining Feature Recognition With Deep Learning. In *Proceedings of the ASME 2020 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference* (Vol. 11A: 46th Design Automation Conference). Virtual, Online. August 17–19, 2020. V11AT11A003. ASME. https://doi.org/10.1115/DETC2020-22355

[6] Wu, R., Xiao, C., & Zheng, C. (2021). DeepCAD: A Deep Generative Network for Computer-Aided Design Models. *ArXiv*. https://arxiv.org/abs/2105.09492

[7] Masson, W., Ranchod, P., (2016). Reinforcement Learning with Parameterized Actions. *ArXiv*. https://arxiv.org/abs/1509.01644

[8] Kochenderfer, Mykel J., et al. Algorithms for Decision Making. Massachusetts Institute of Technology, 2022.

[9] Radford, A., Metz, L., & Chintala, S. (2015). Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks. *ArXiv*. https://arxiv.org/abs/1511.06434